

Lecturer: Yetmwork Tesfaye

MSc. in Computer Science and Engineering

College of Computing and Informatics

Department of Computer Science

Chapter 4: Communication

Objectives

At the end of this Chapter Students be able to know:-

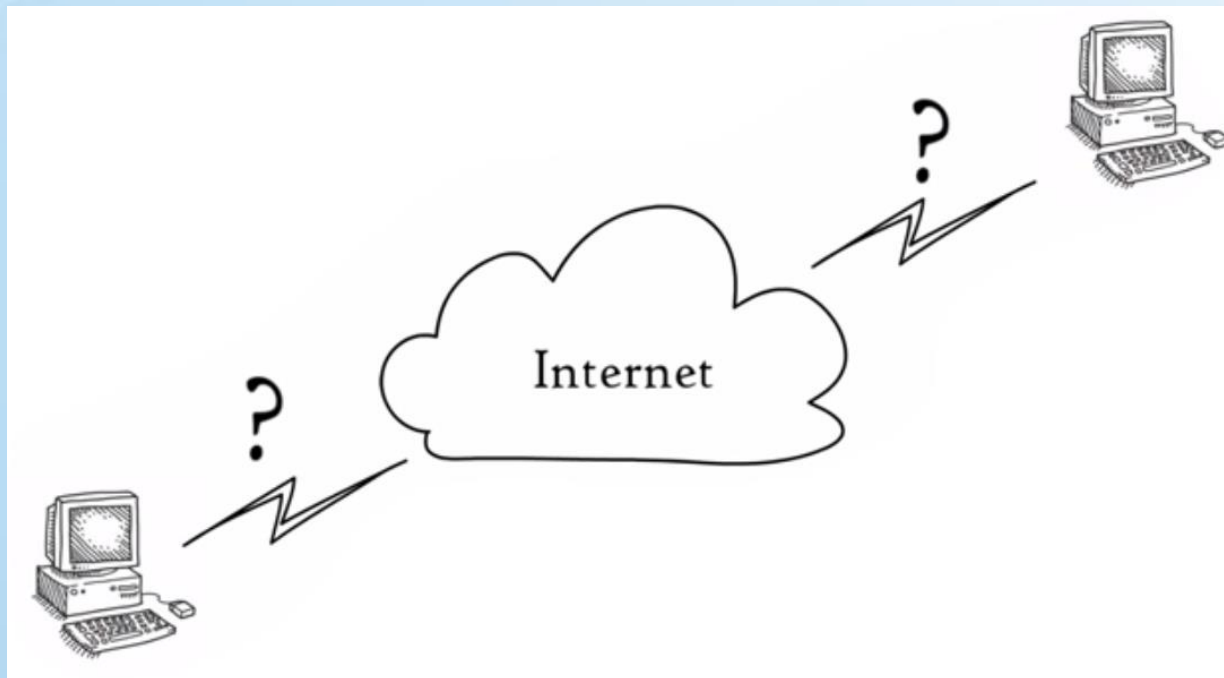
- Layer protocols
- Types of Communication
- Remote Procedure Call

Introduction: What is Communication?

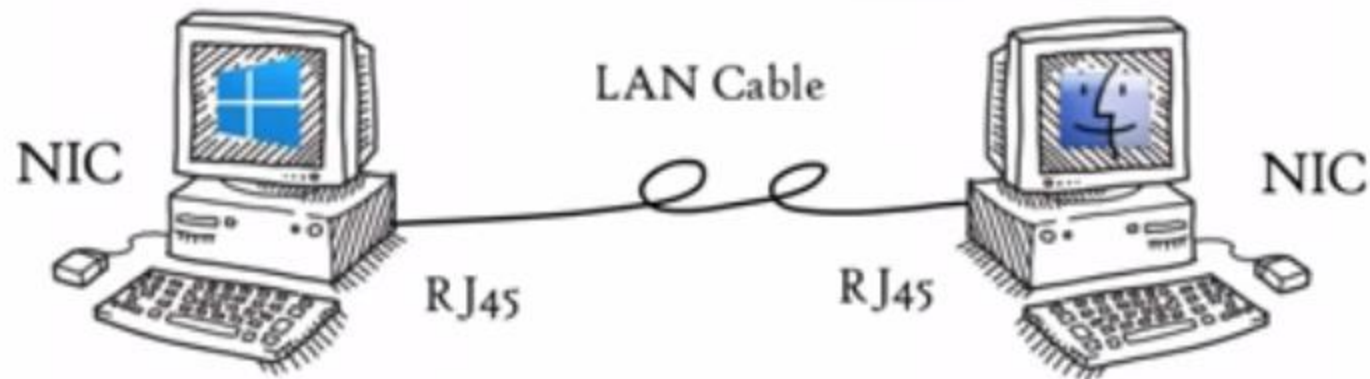
- In distributed system, Communication is Exchange of information between processes on different machines.
- Communication between processes can be
 - **Unstructured Communication**
 - Use shared memory or shared data structure
 - **Structured Communication**
 - Use Explicit messages

Communication Protocols: OSI Model

- OSI model Defines how data is transferred from one computer to another computer in computer network

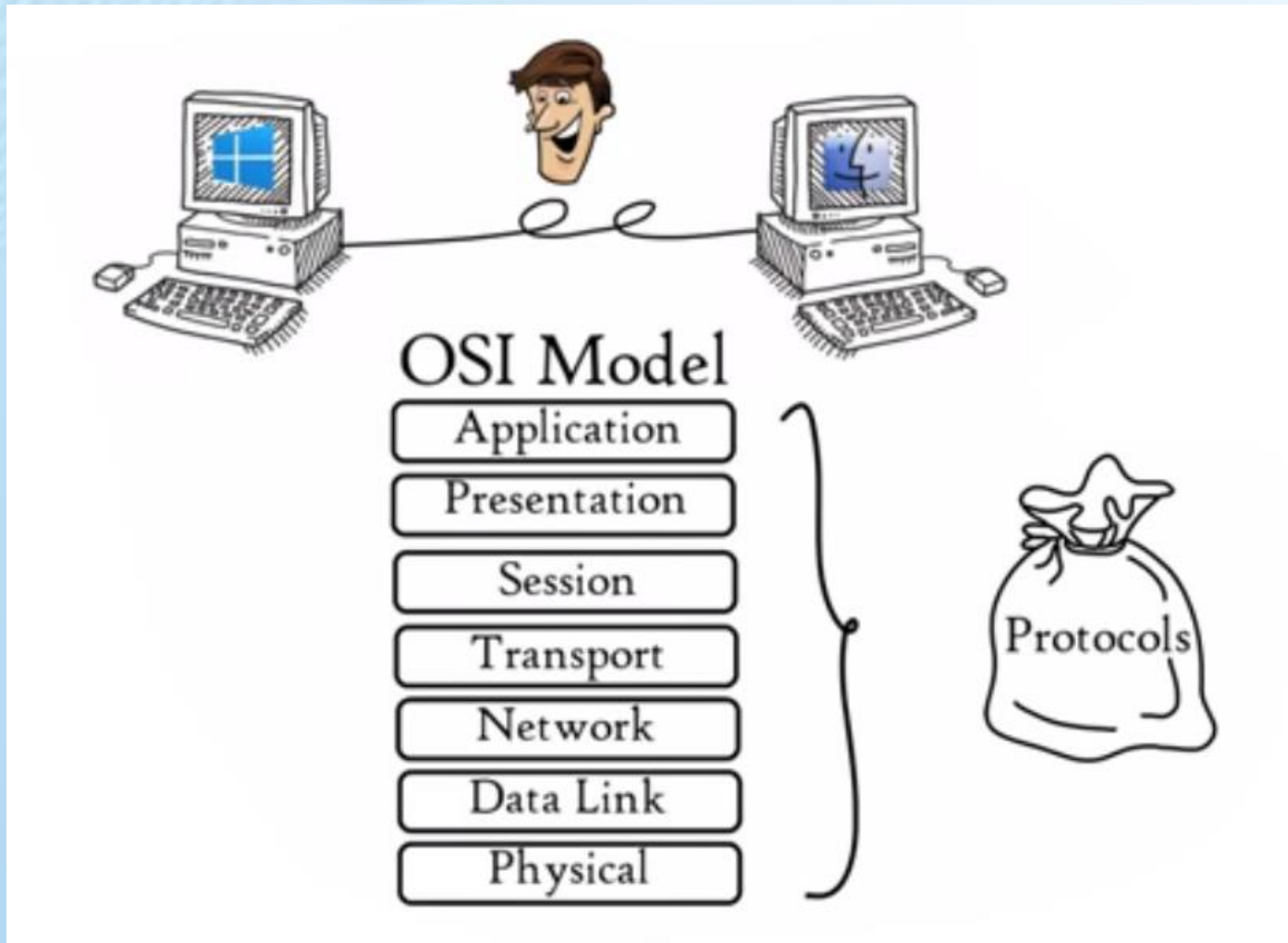


Communication Protocols: OSI Model



Communication Protocols: OSI Model

- In order to accomplish successful communication between computers, 7 layers OSI models is introduced by ISO in 1984.



Communication Protocols: Layered Protocols

- Protocols are agreements/rules on communication
- Protocols could be connection oriented or connectionless.

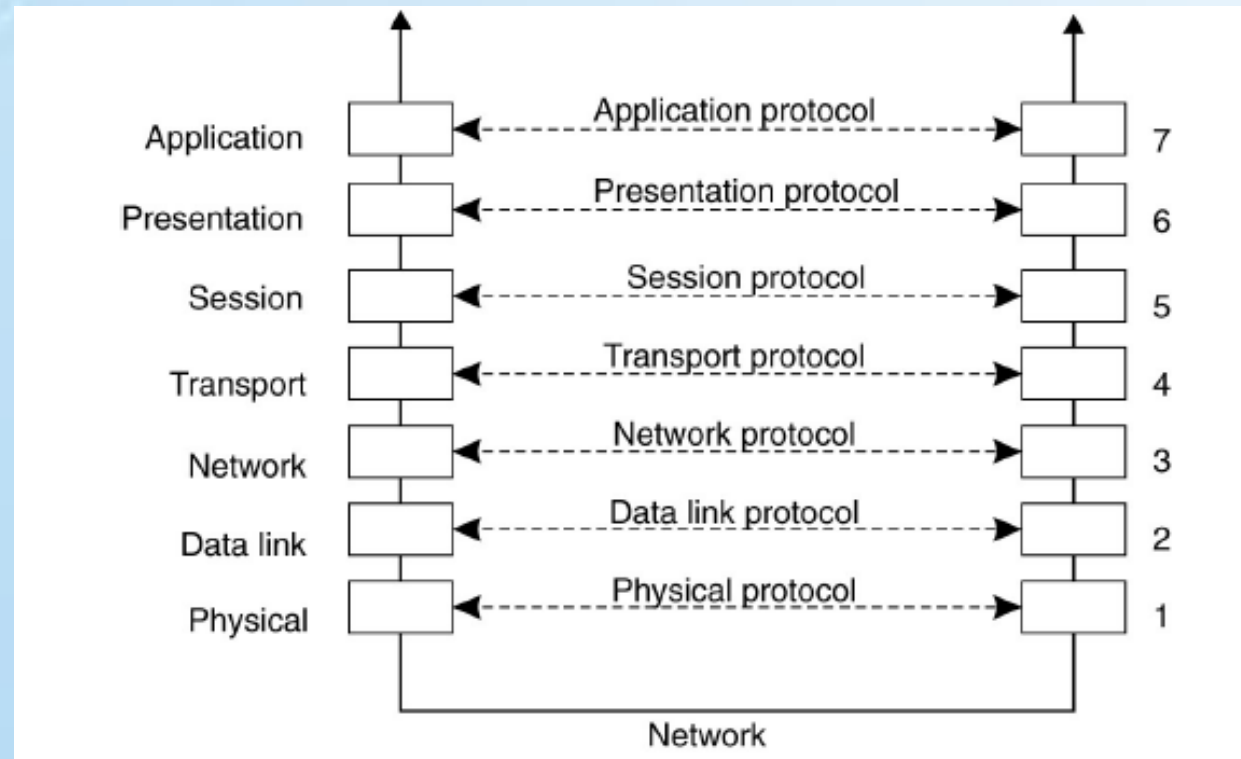


Figure :Layers, interfaces, and protocols in the OSI model.

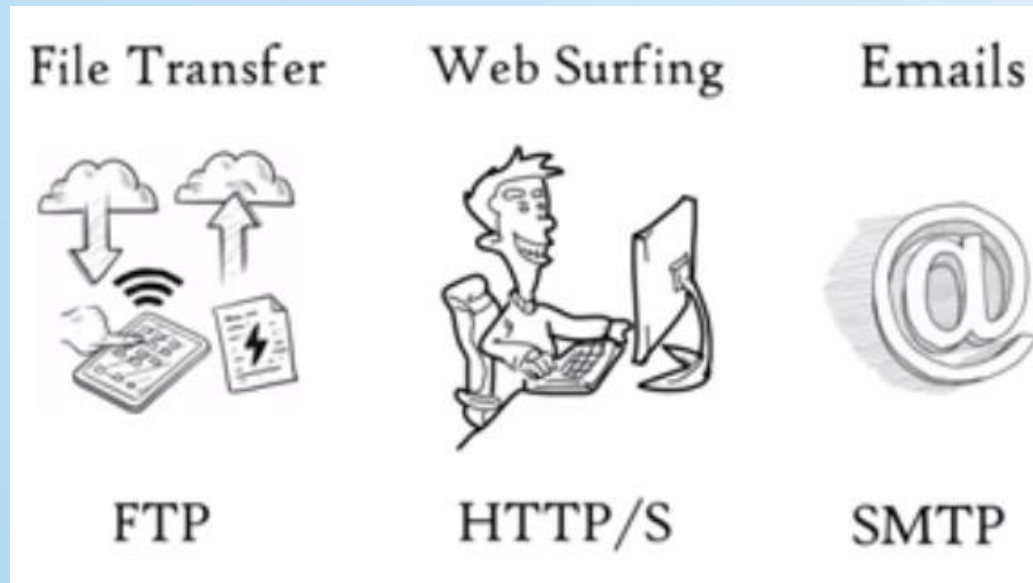
Communication Protocols: Layered

Protocols: 1.Application Layer

- Used by network applications



- There are various types of protocols used in application layers



Communication Protocols: Layered Protocols:

2.Presentation Layer

- Receives data from Application Layer.
- The data is always received in the form of character and numbers.
- Converts a data to machine understandable binary format.
- Mainly Presentation Layer performs 3 important tasks.
- Translation, Data compression, Encryption/Decryption

Communication Protocols: Layered Protocols: Presentation Layer(Translation)

Application



Presentation

DFGJUYRG
64578995412

100011101101

eg. ASCII → EBCDIC

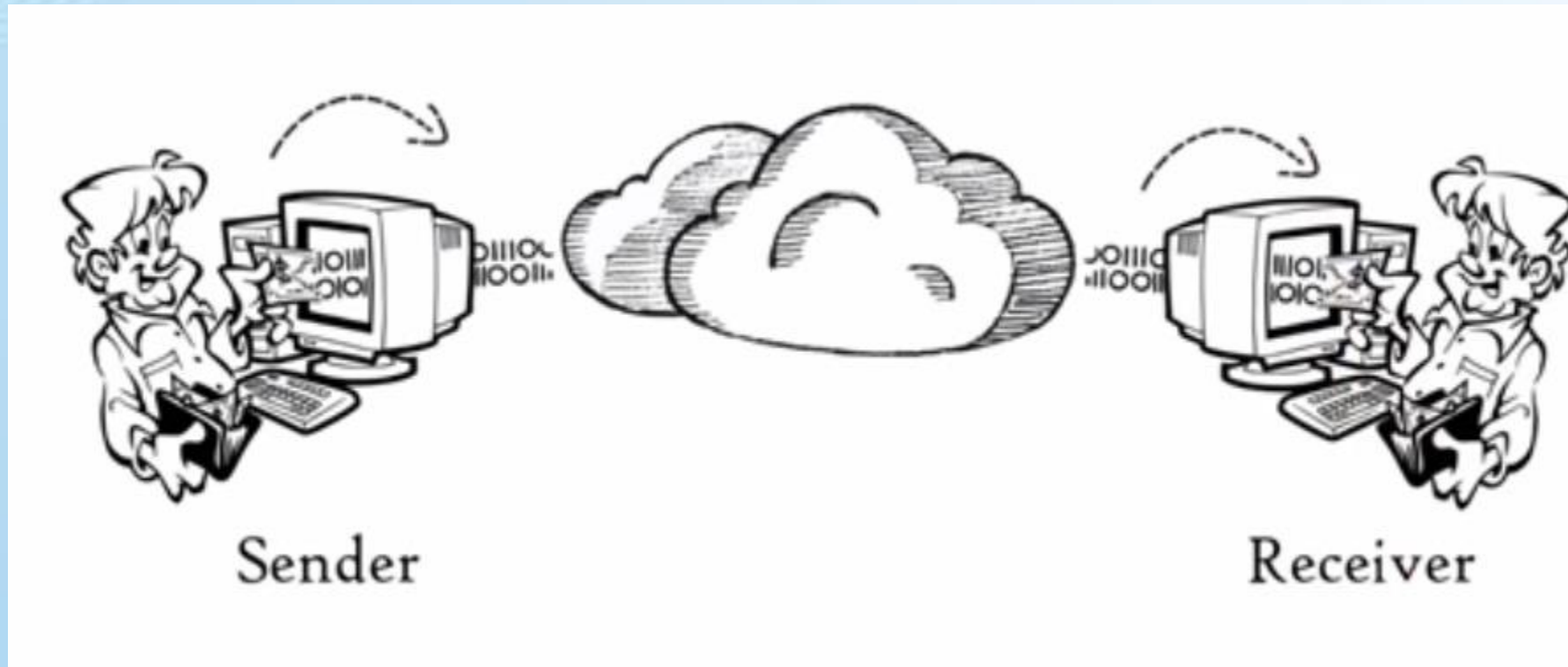
Communication Protocols: Layered Protocols: Presentation Layer(Data Compression)

- Reduce size of data, Can be lossy and Lossless
- **Lossy** - by permanently removing some of the original data.
- **Lossless** - by removing unnecessary metadata.



Communication Protocols: Layered Protocols: Presentation Layer(Encryption/Decryption)

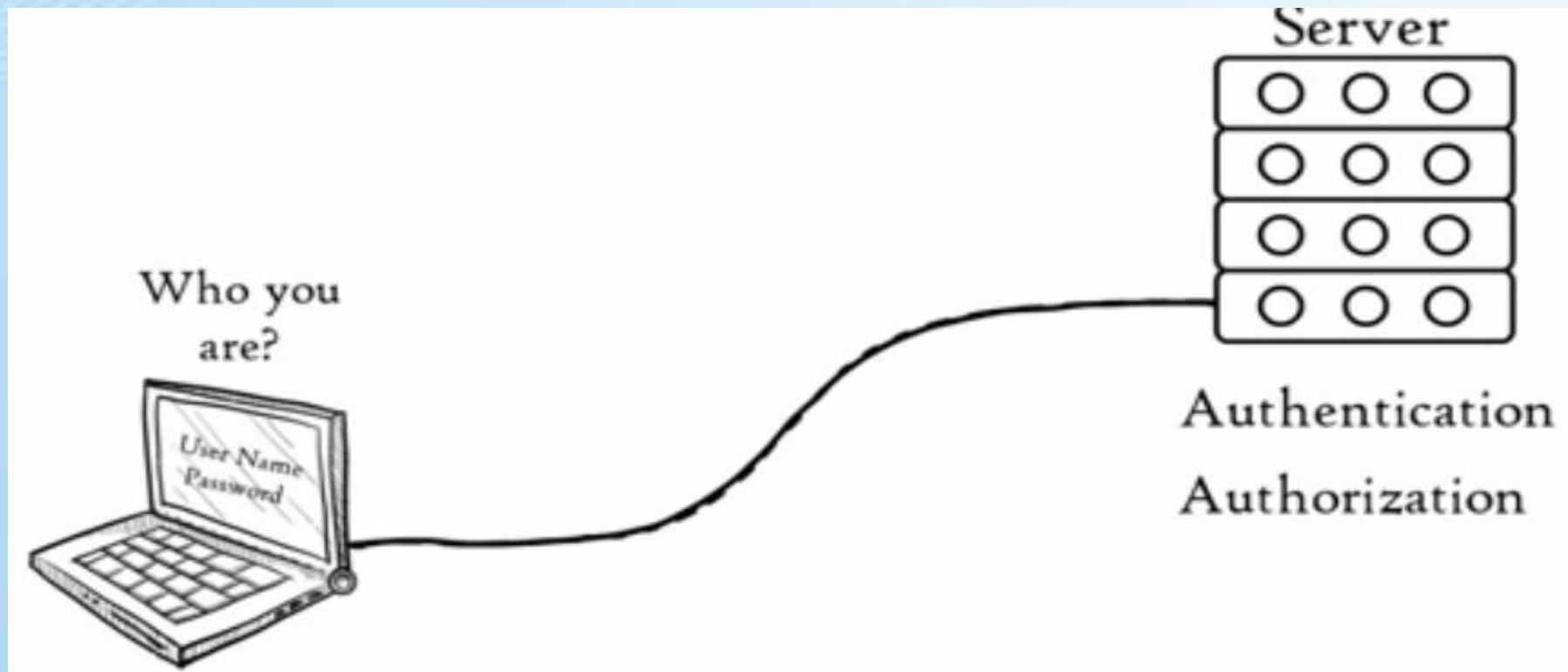
- Ensure integrity of data
- SSL(Secure Sockets Layer) used for encryption and decryption



Communication Protocols: Layered Protocols:

3.Session Layer

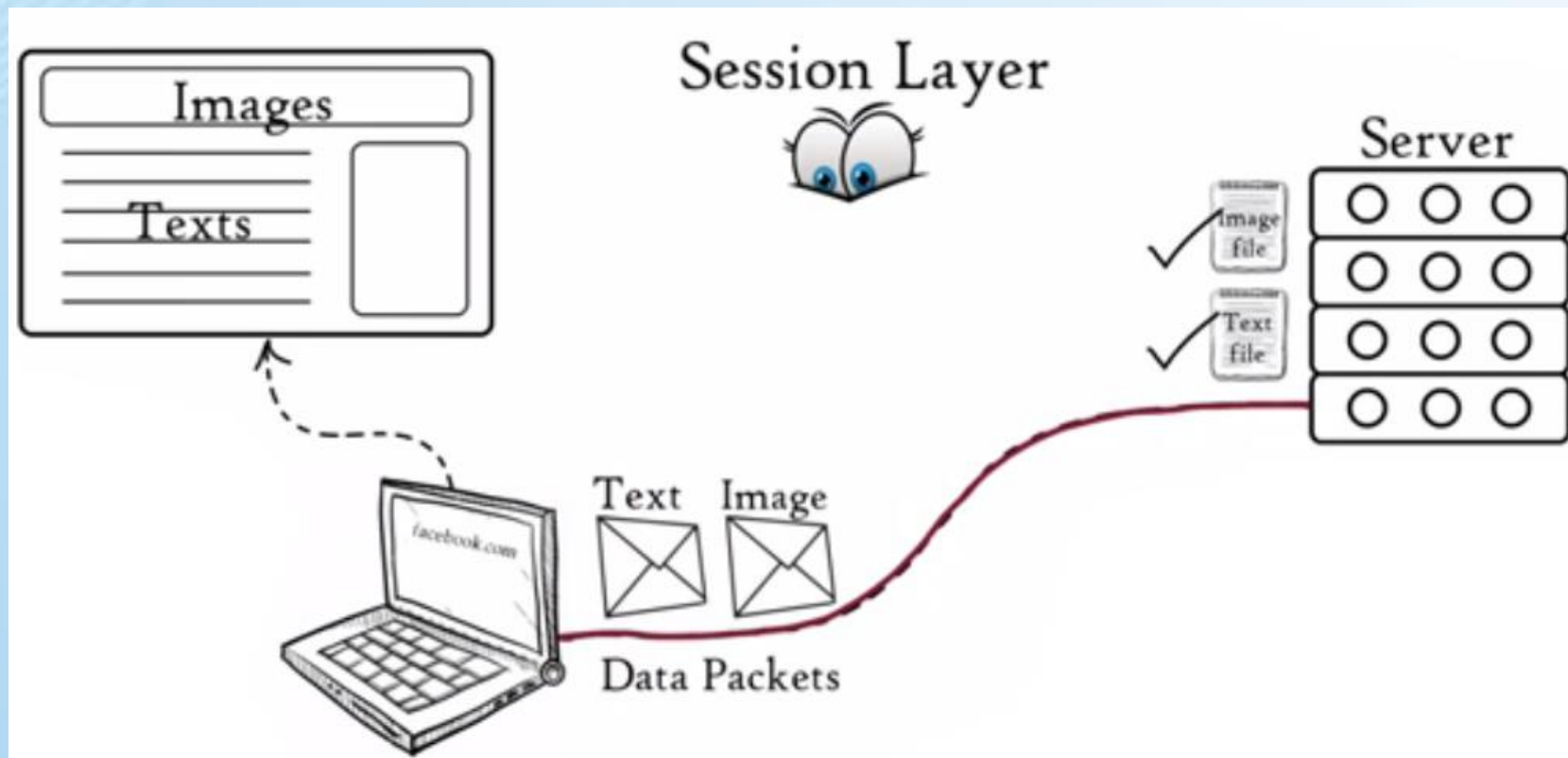
- Keeps track of who is transmitting.
- Enable sending and receiving of data and termination of connection(session).
- Authentication ,Authorization and Session Management



Communication Protocols: Layered Protocols:

3.Session Layer(Session Management)

- Which packet belongs to which file and tracks where the received data packets go.
- APIs and NETBIOS



Communication Protocols: Layered Protocols:

4.Transport Layer

- Controls the Reliability of Communication



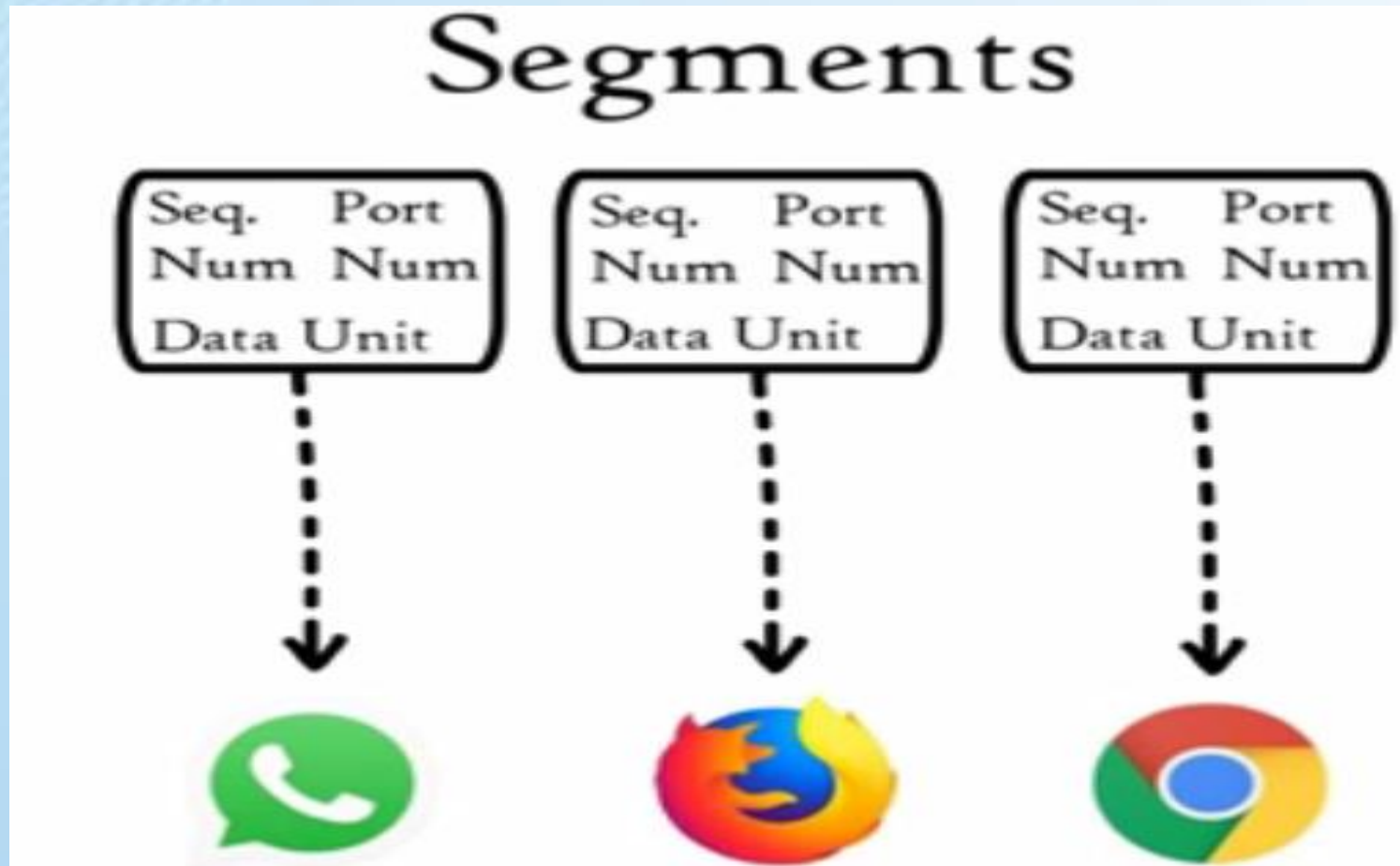
Segmentation
Flow Control
Error Control



Communication Protocols: Layered Protocols:

4.Transport Layer(segmentation)

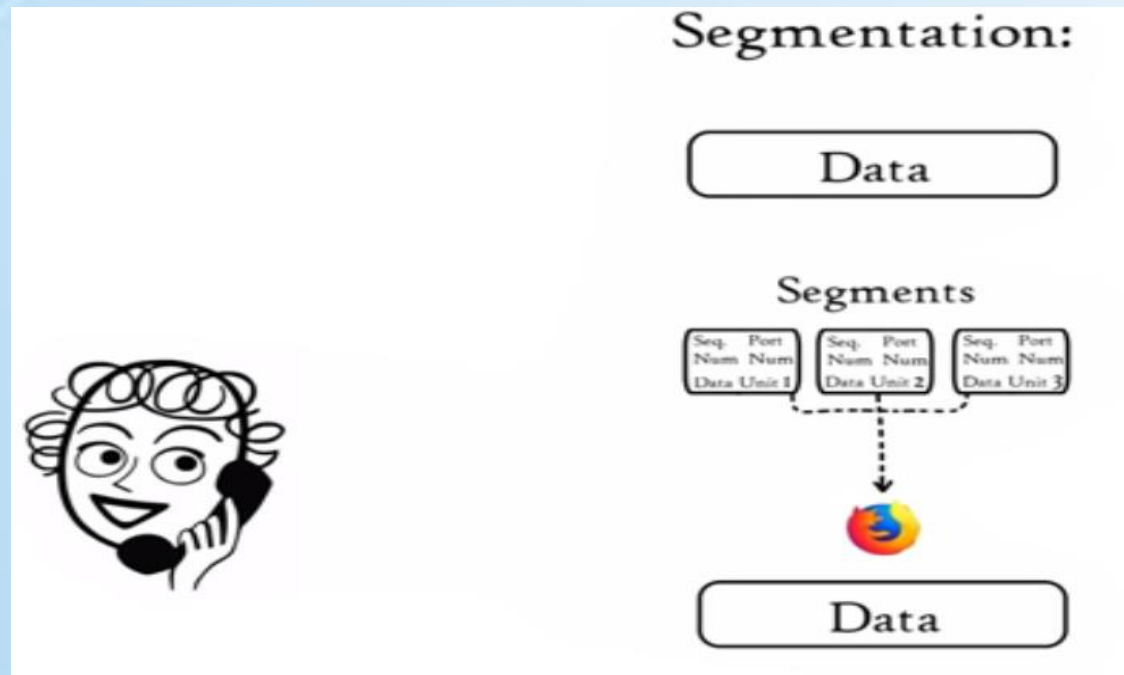
- Data is divided into small data units called segment.



Communication Protocols: Layered Protocols:

4.Transport Layer(segmentation)

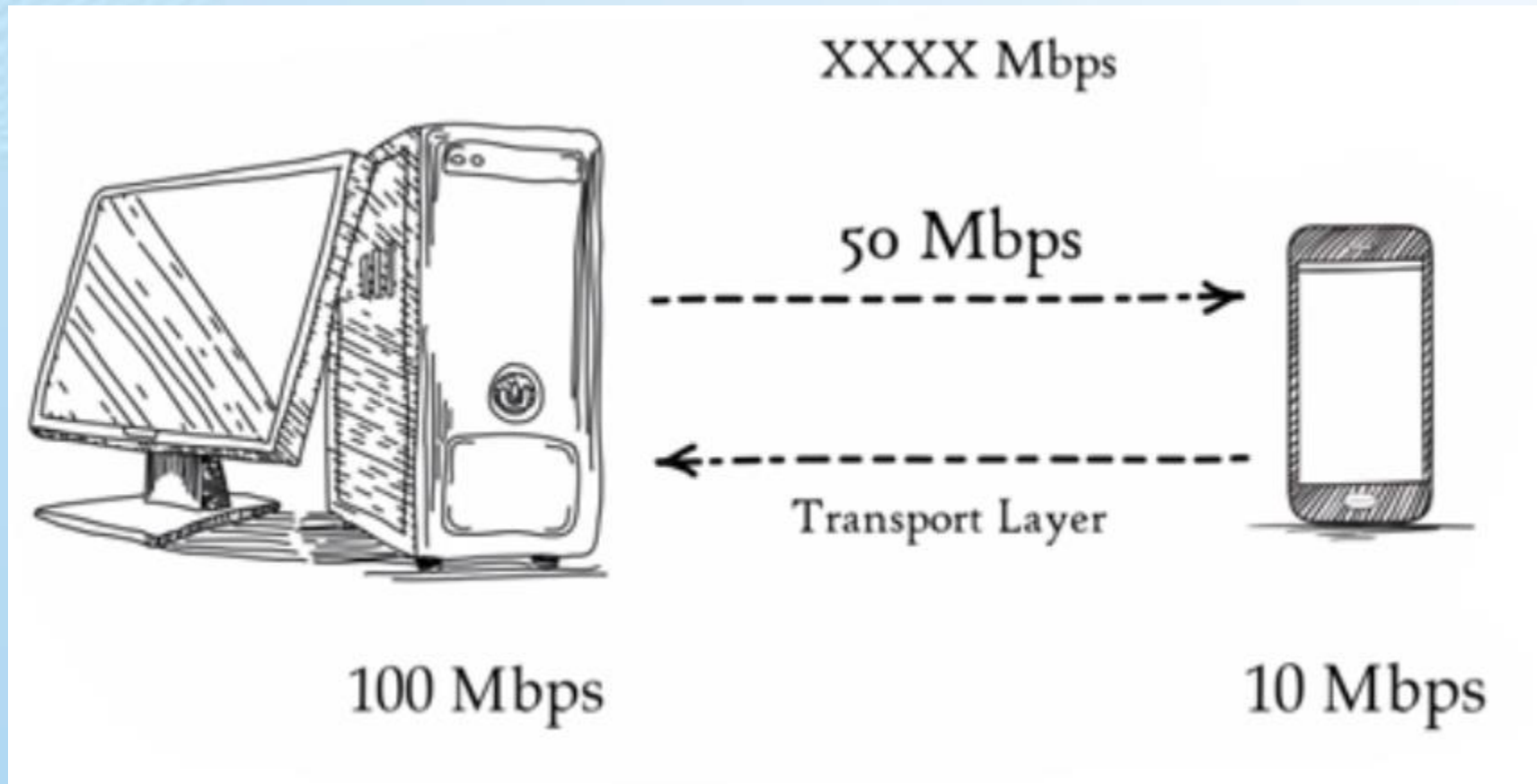
- Sequence number helps to reassemble segments in correct order to form correct message at receiver side



Communication Protocols: Layered Protocols:

4. Transport Layer (Flow Control)

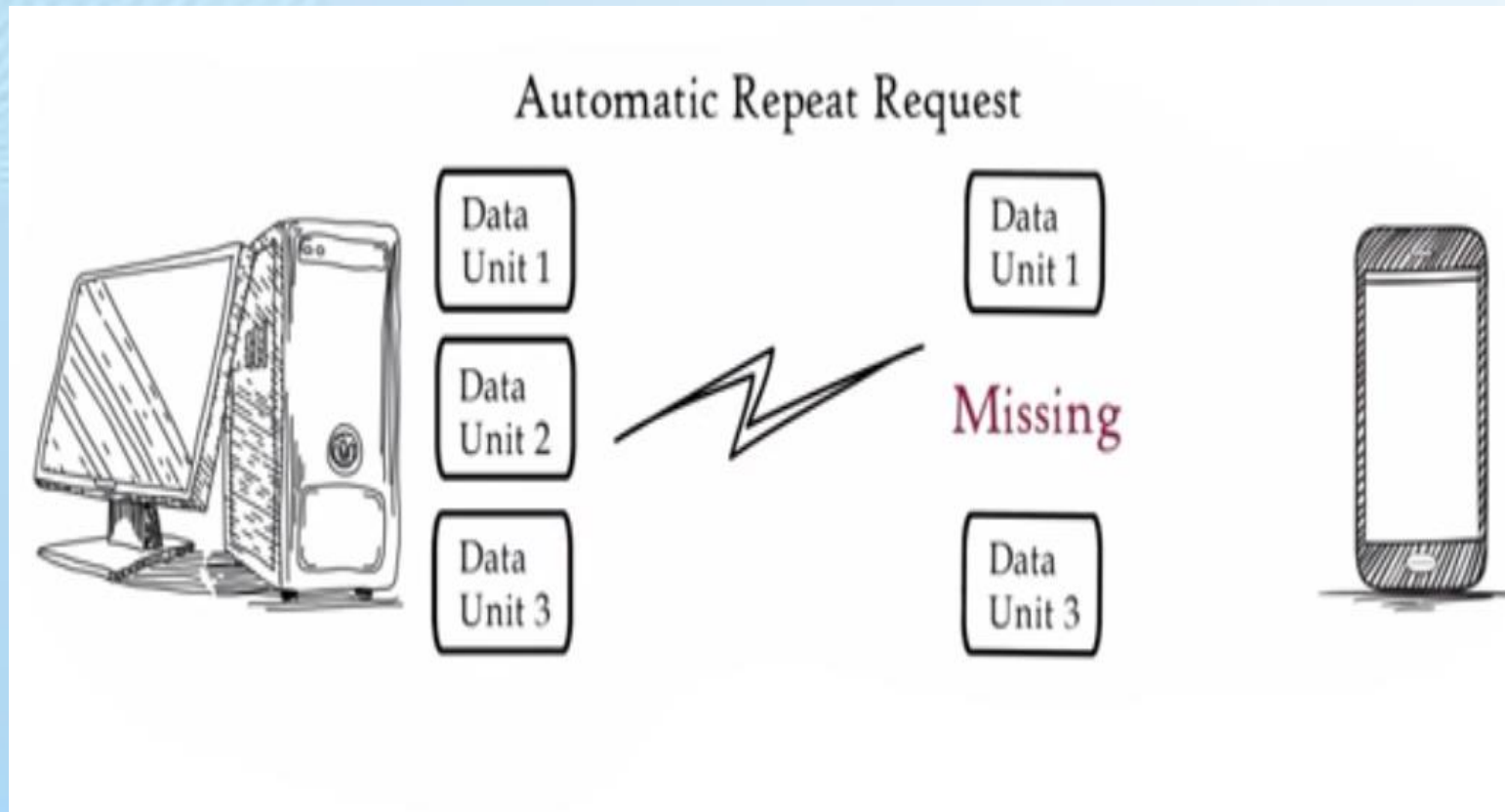
- Controls the amount of data being transmitted



Communication Protocols: Layered Protocols:

4. Transport Layer(Error Control)

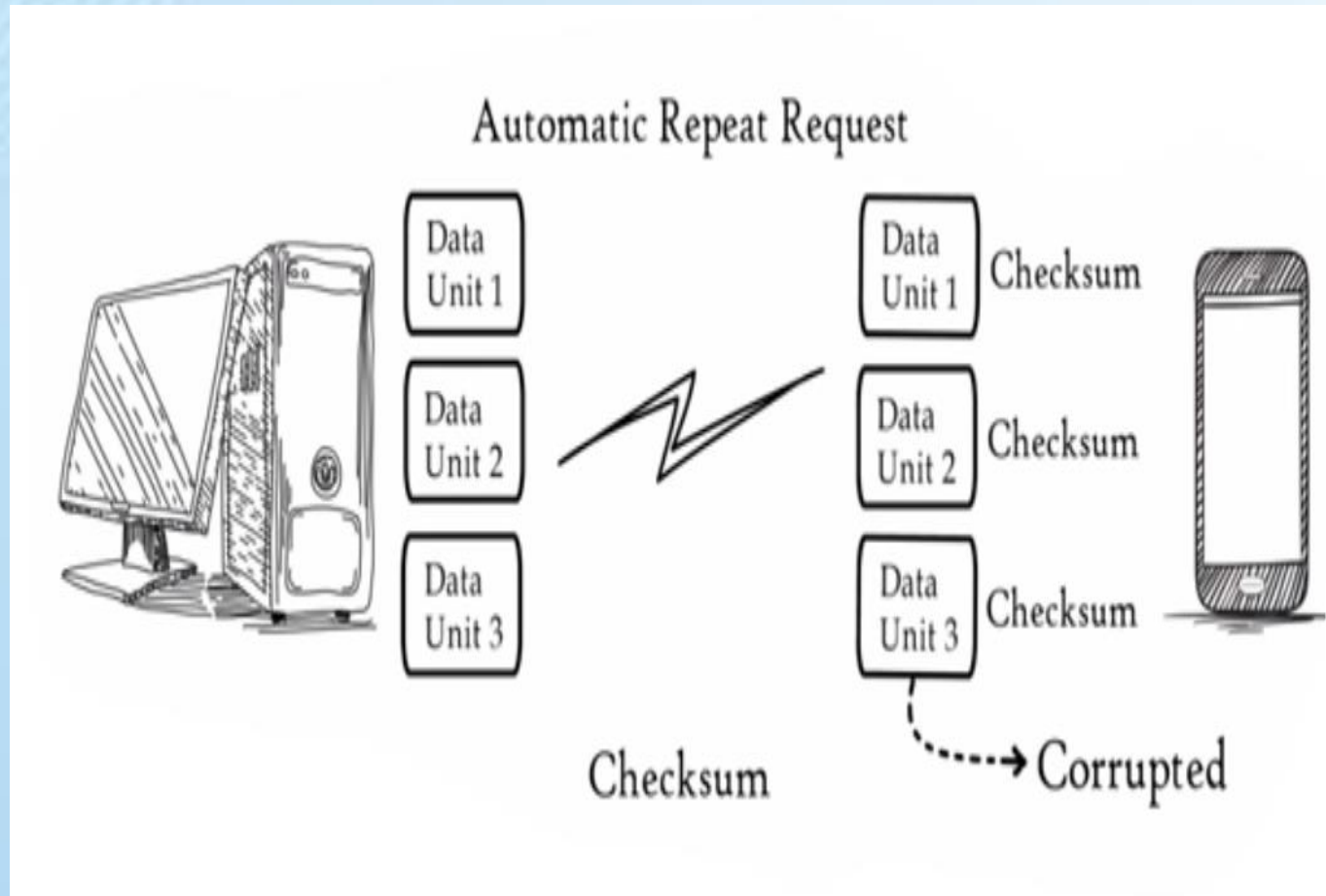
- Automatic Repeat request-lost data can be retransmitted



Communication Protocols: Layered Protocols:

4. Transport Layer(Error Control)

- Protocols of Transport Layers are TCP and UDP



Communication Protocols:

4. Transport Layer (Connection and connectionless transmission)

UDP



No feedback

TCP

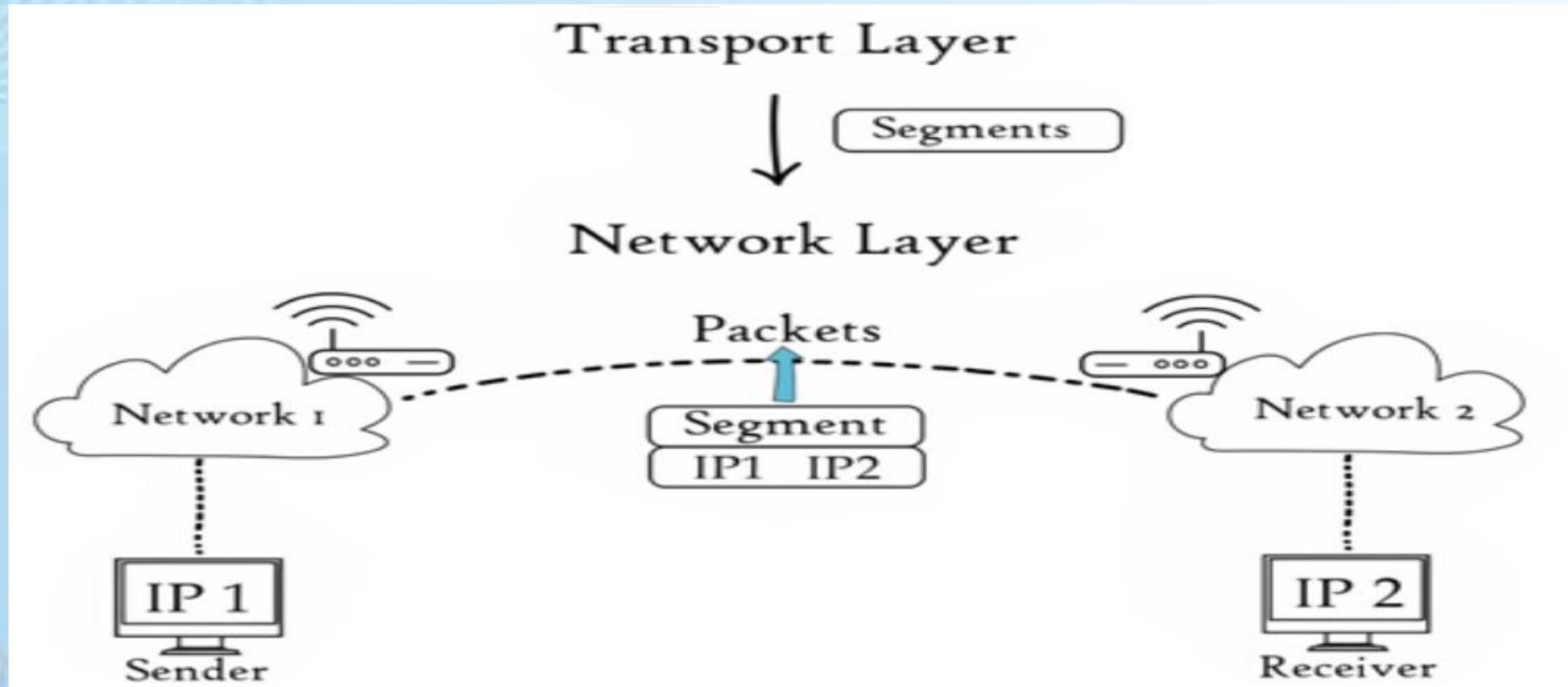


Feedback

Communication Protocols: Layered Protocols:

5. Network Layer

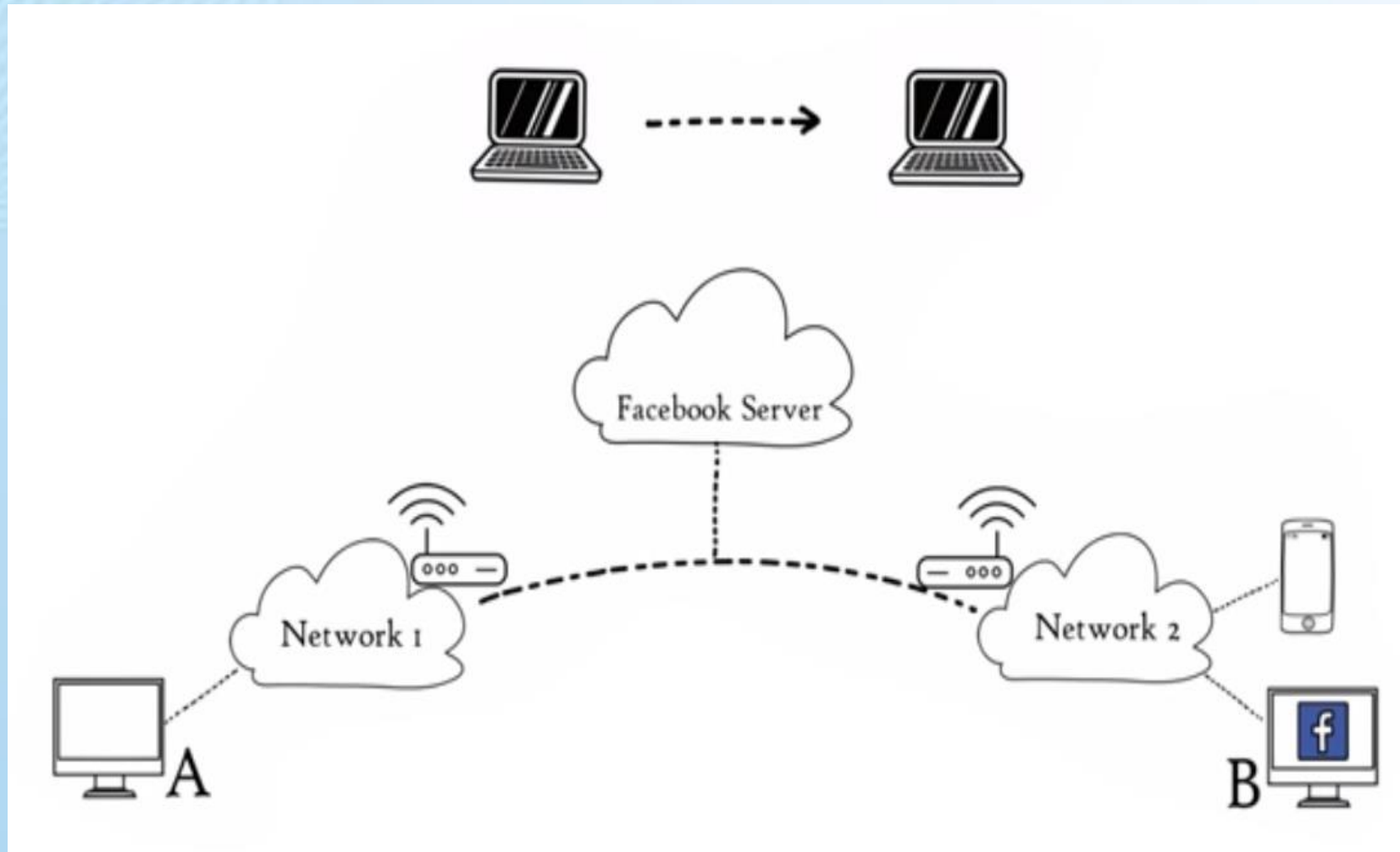
- Layer where router resides
- logical addressing, path determination and Routing



Communication Protocols: Layered Protocols:

5. Network Layer(Routing)

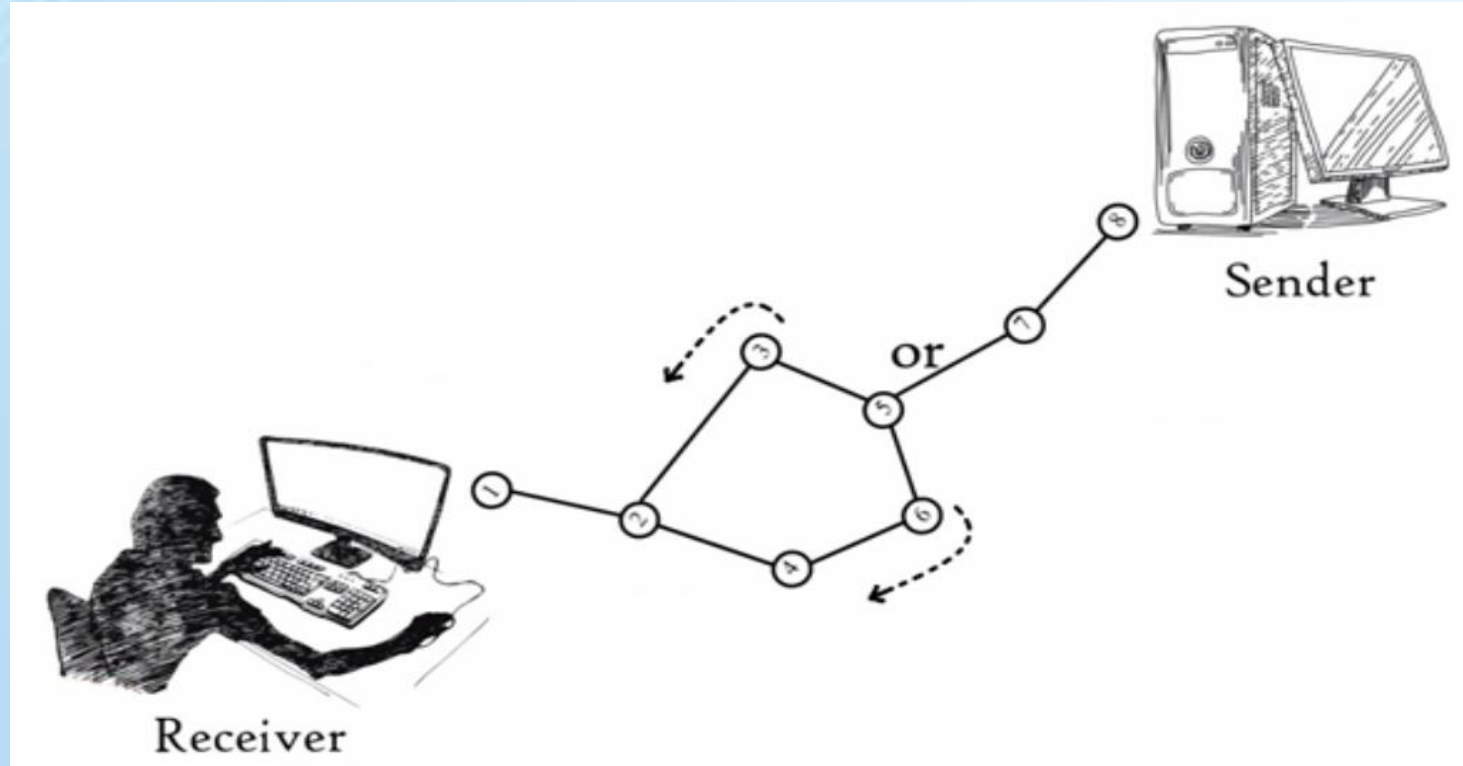
- Moving data from sender to destination



Communication Protocols: Layered Protocols:

5. Network Layer (Path determination)

- Choose best possible path to deliver packets
- Protocol : IP



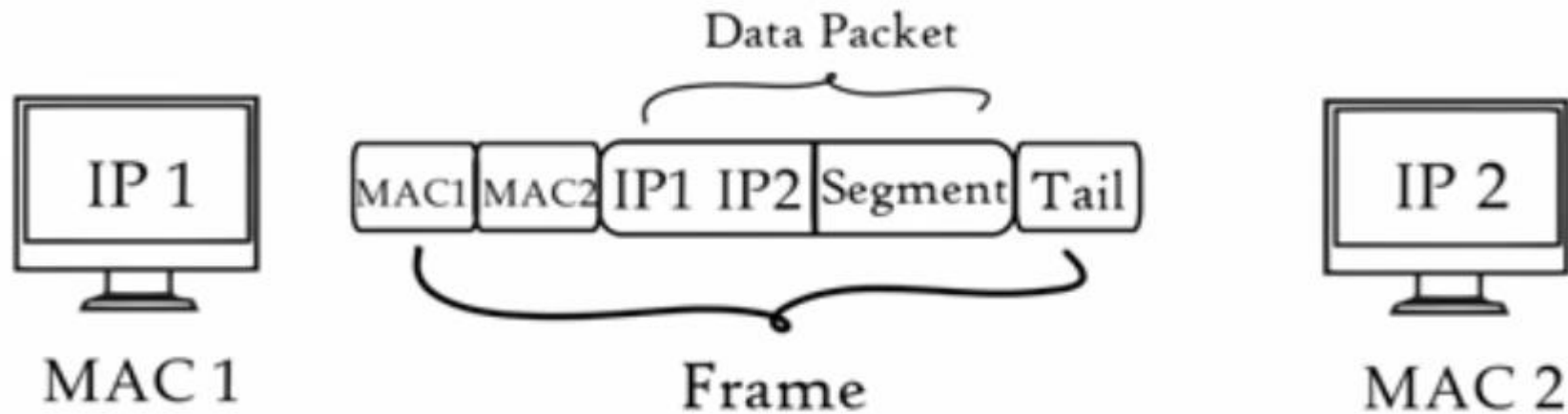
Communication Protocols: Layered Protocols:

6. Data Link Layer

- Access the media and error detection

- Logical addressing : Network layer

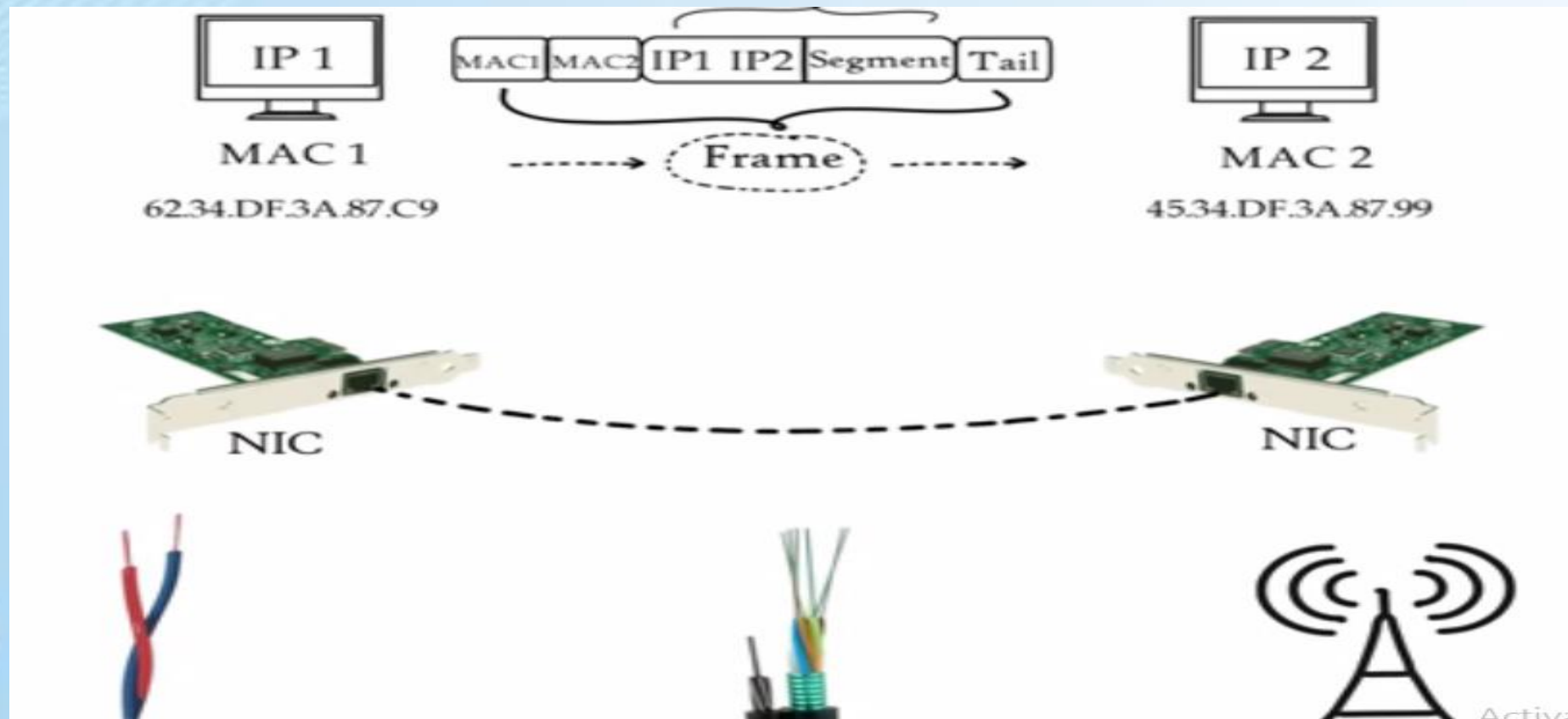
- Physical addressing : Data Link layer



Communication Protocols: Layered Protocols:

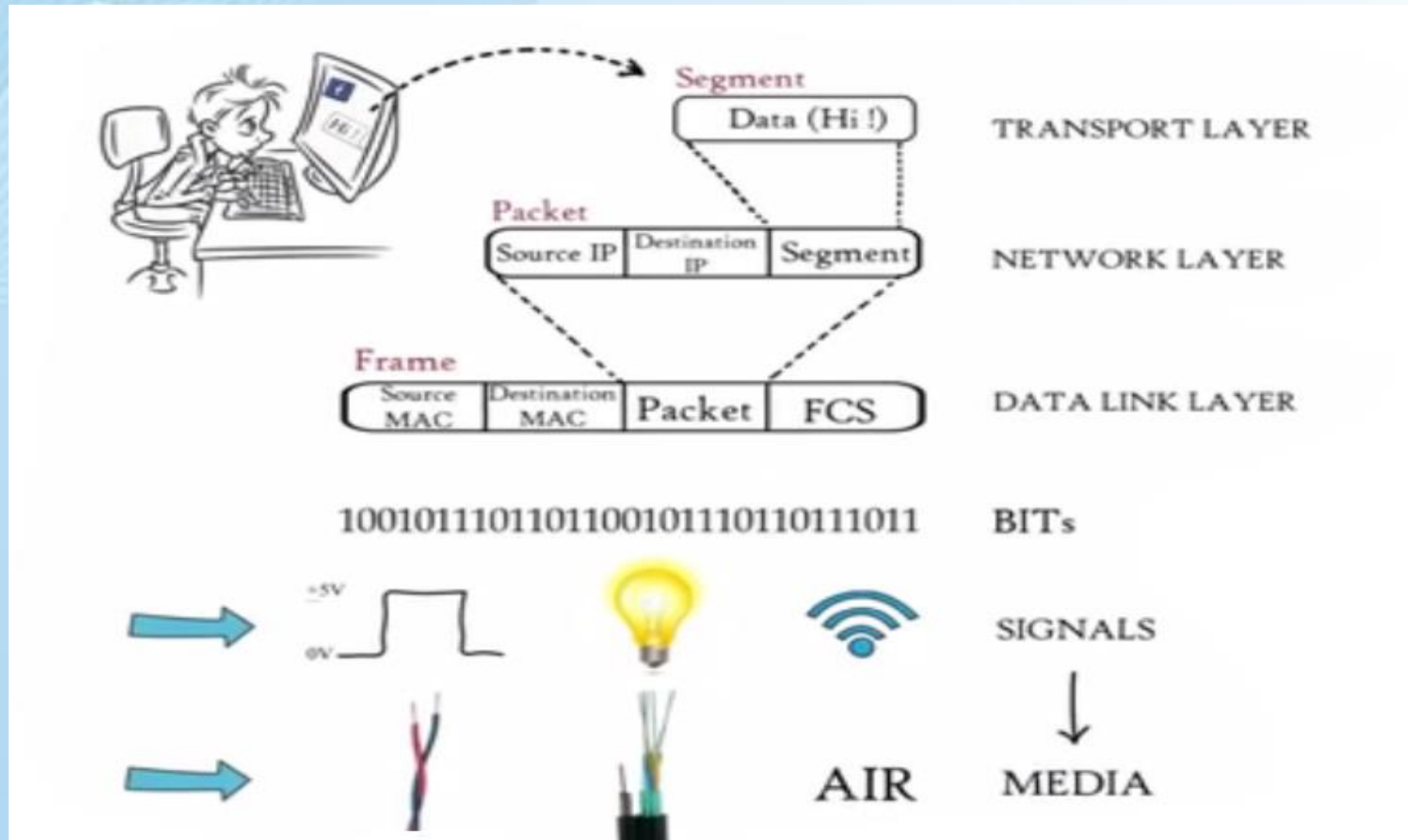
6.Data Link Layer

- Embedded as software in NIC.
- Provide a means to transfer data from one computer to another via local media.



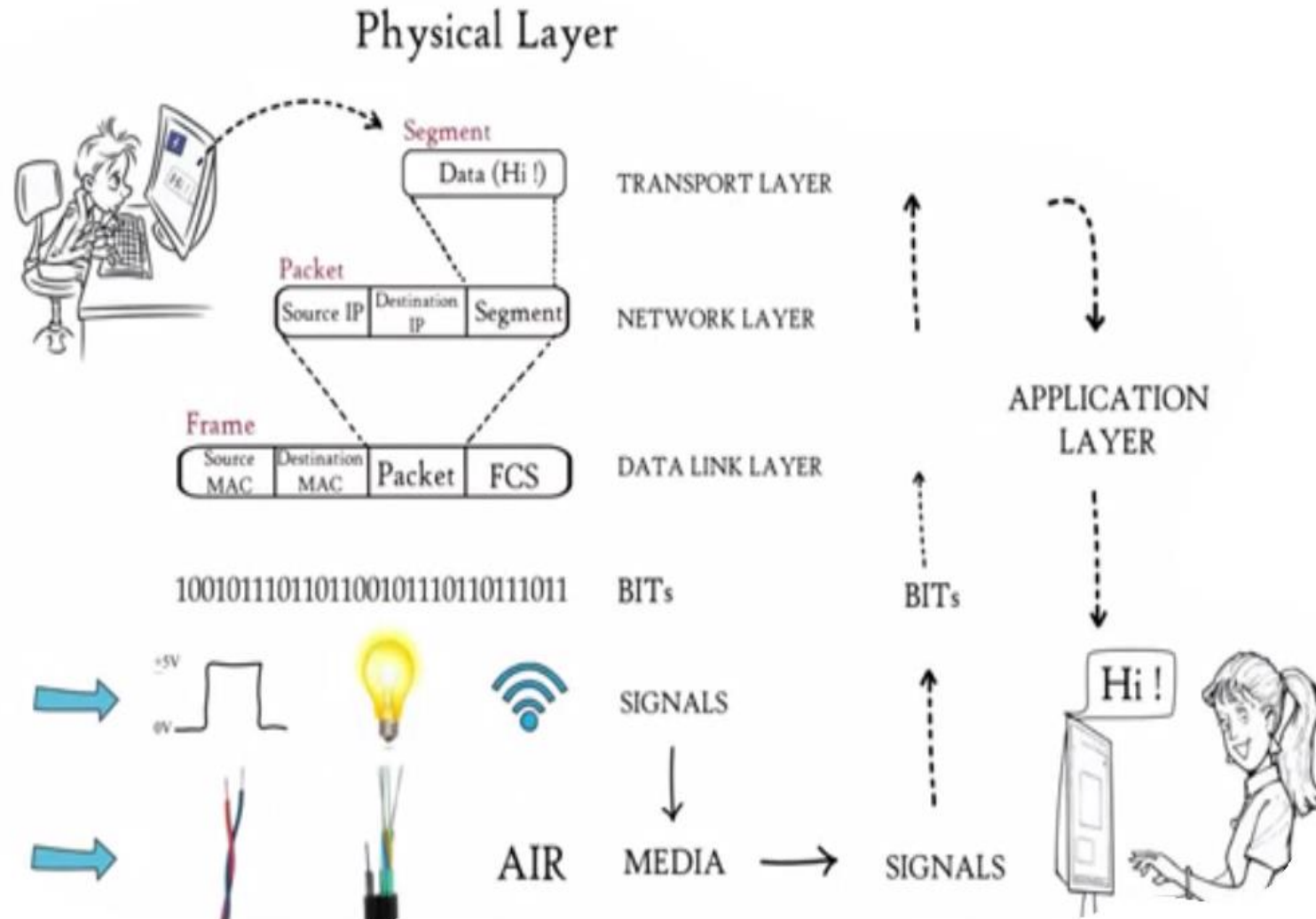
Communication Protocols: Layered Protocols:

7. Physical Layer



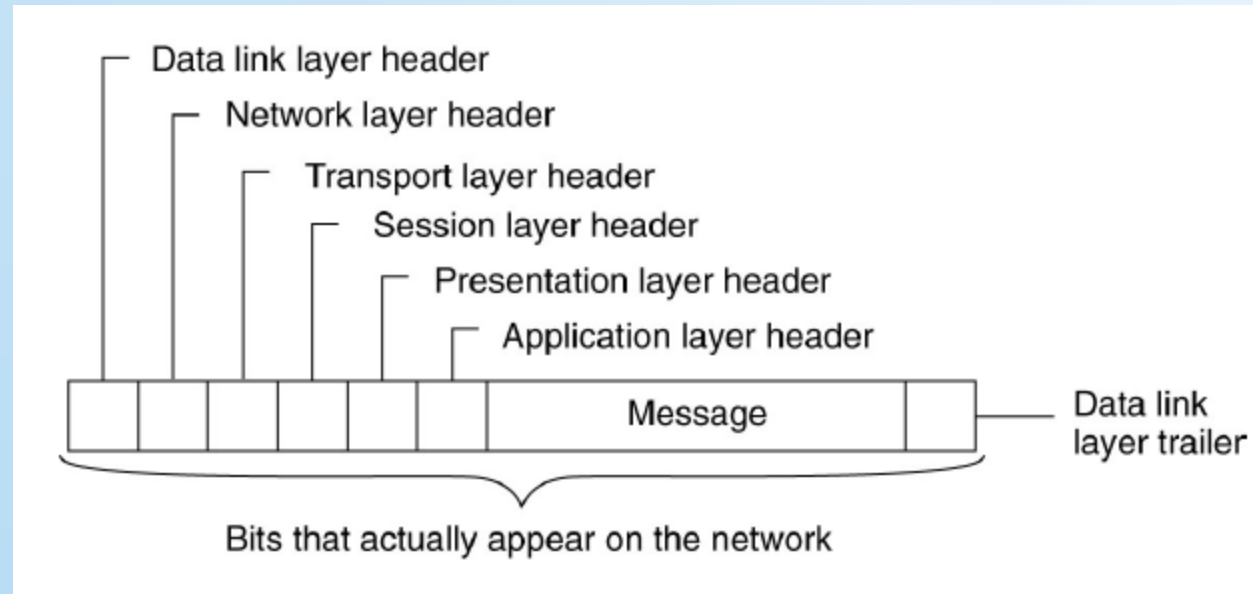
Communication Protocols: Layered Protocols:

7. Physical Layer



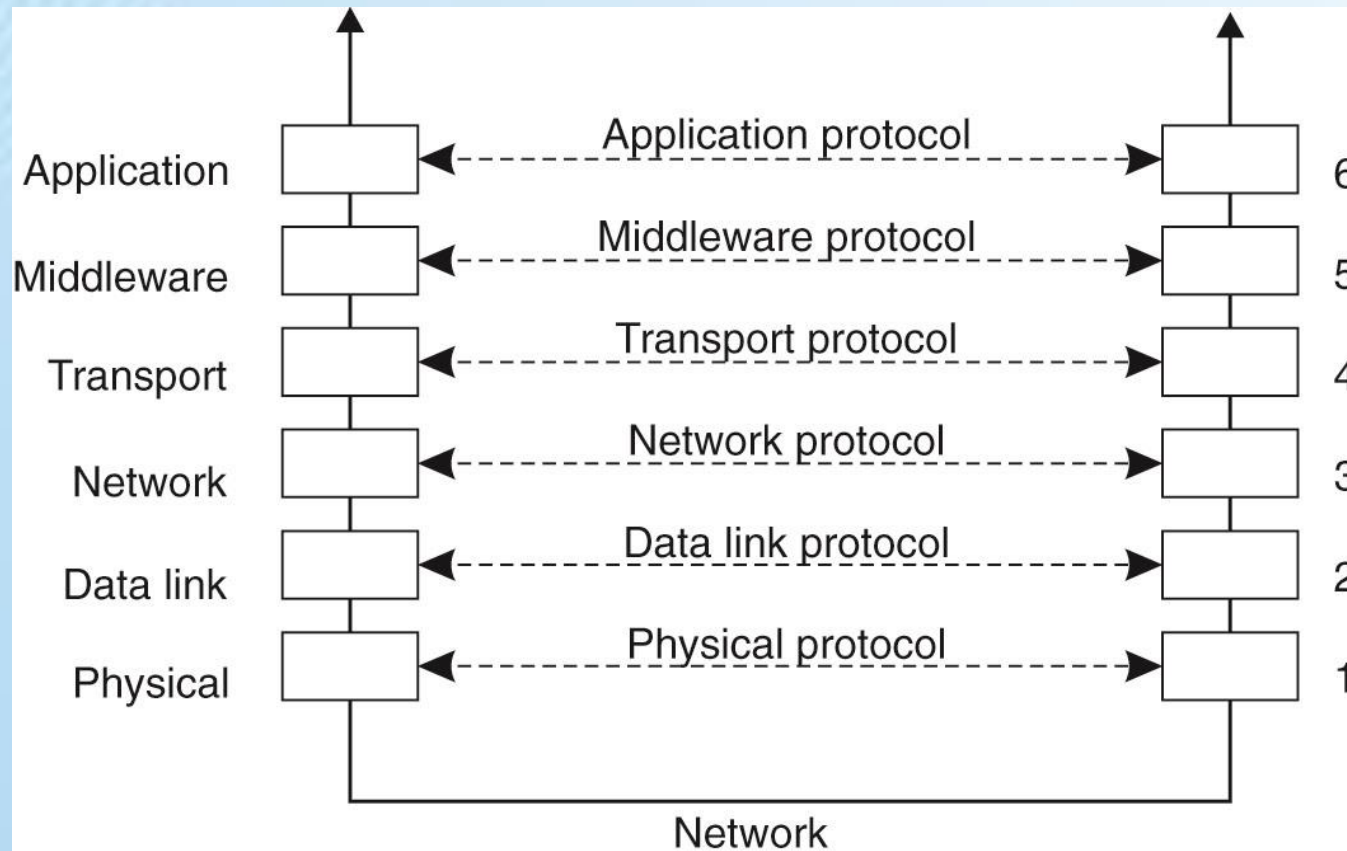
Communication Protocols: Layered Protocols

- A typical message as it appears on the network



Middleware Protocols

- Middleware: layer that resides between an OS and application
 - May implement general purpose protocol.



An adapted reference model for networked communication.

Push or Pull architecture

- Two communication models for clients and servers to communicate.
- Client-pull architecture
 - Clients pull data from servers(by sending requests)
 - E.g. HTTP
- Server-push architecture
 - Servers push data to clients
 - E.g Video Streaming

Types of Communication

- Persistent versus transient
- Synchronous versus asynchronous
- Discrete versus streaming

Persistent versus Transient Communication

- **Persistent:** messages are held by the middleware communication service until they can be delivered. (Think email)
 - Sender can terminate after executing send
 - Receiver will get message next time it runs
- **Transient:** Messages exist only while the sender and receiver are running
 - Communication errors or inactive receiver cause the message to be discarded.
 - If the router can't forward the message it will drop it.

Asynchronous versus Synchronous Communication

- **Asynchronous:** (non-blocking) sender resumes execution as soon as the message is passed to the communication/middleware software.
 - Message is buffered temporarily by the middleware until sent/received
- **Synchronous:** sender is blocked until
 - The OS or middleware notifies acceptance of the message, *or*
 - The message has been delivered to the receiver, *or*
 - The receiver processes it & returns a response.

Synchronous Communication

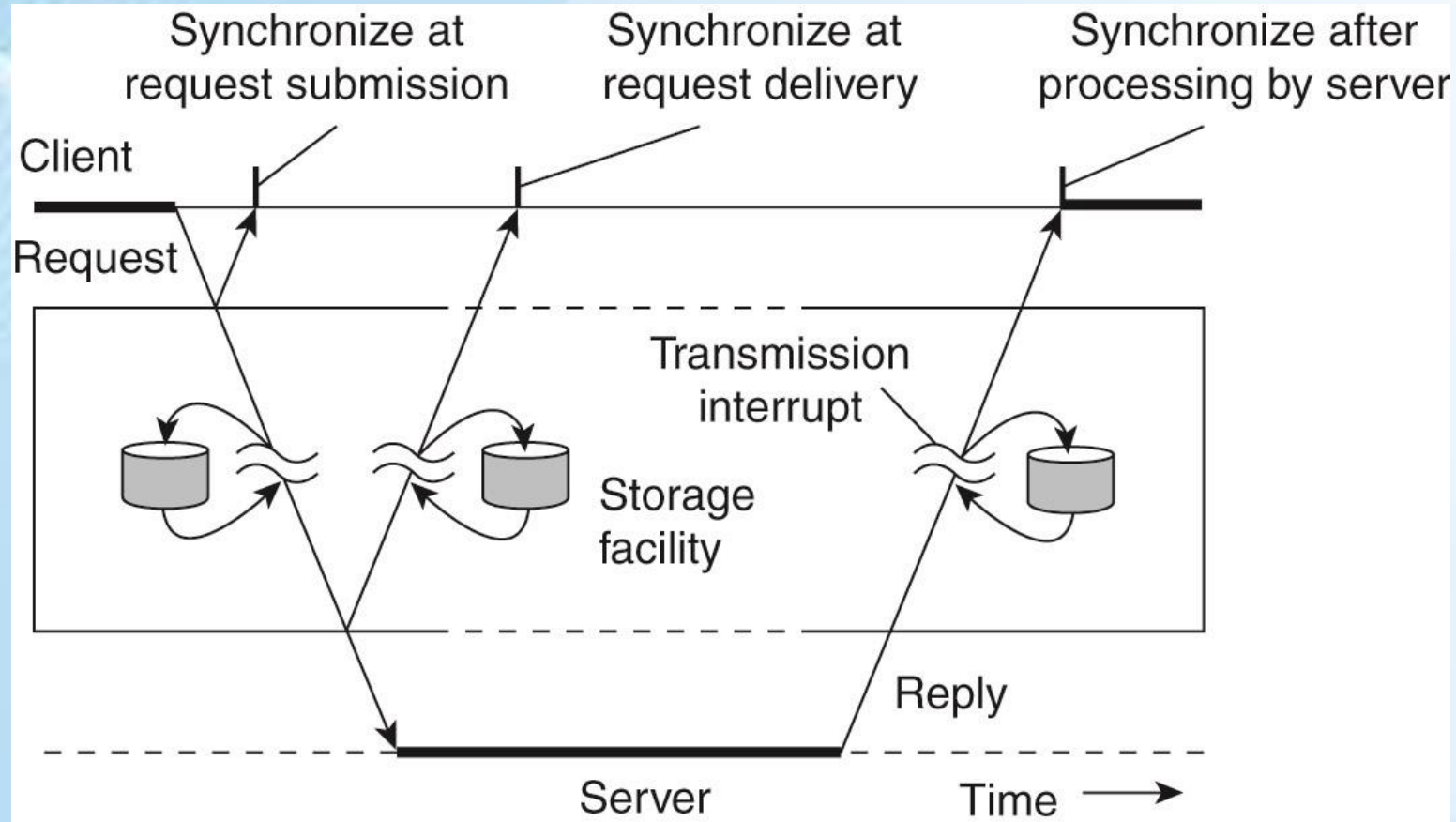


Figure 4-4. Viewing middleware as an intermediate (distributed) service in application-level communication.

Discrete versus Streaming Communication

- **Discrete:** communicating parties exchange discrete messages
 - The sender sends one message, and the receiver receives and processes that specific message before moving on to the next one.
- **Streaming:** one-way communication; a “session” consists of multiple messages from the sender that are related either by send order etc.
 - In streaming, the sender continuously sends a stream or sequence of messages to the receiver without waiting for individual acknowledgments or responses for each message.

Remote Procedure Call

- Allow remote services to be called as procedures
- make distributed computing look like centralized computing.
- Codes for procedures reside in another machine(not the machine just called)
- When a process on machine A calls' a procedure on machine B, the calling process on A is suspended and execution of the called procedure takes place on B.

Remote Procedure Call

- Information can be transported from the caller to the callee in the parameters and can come back in the procedure result.
- No message passing at all is visible to the programmer.
- Allow location transparency

Example of RPC

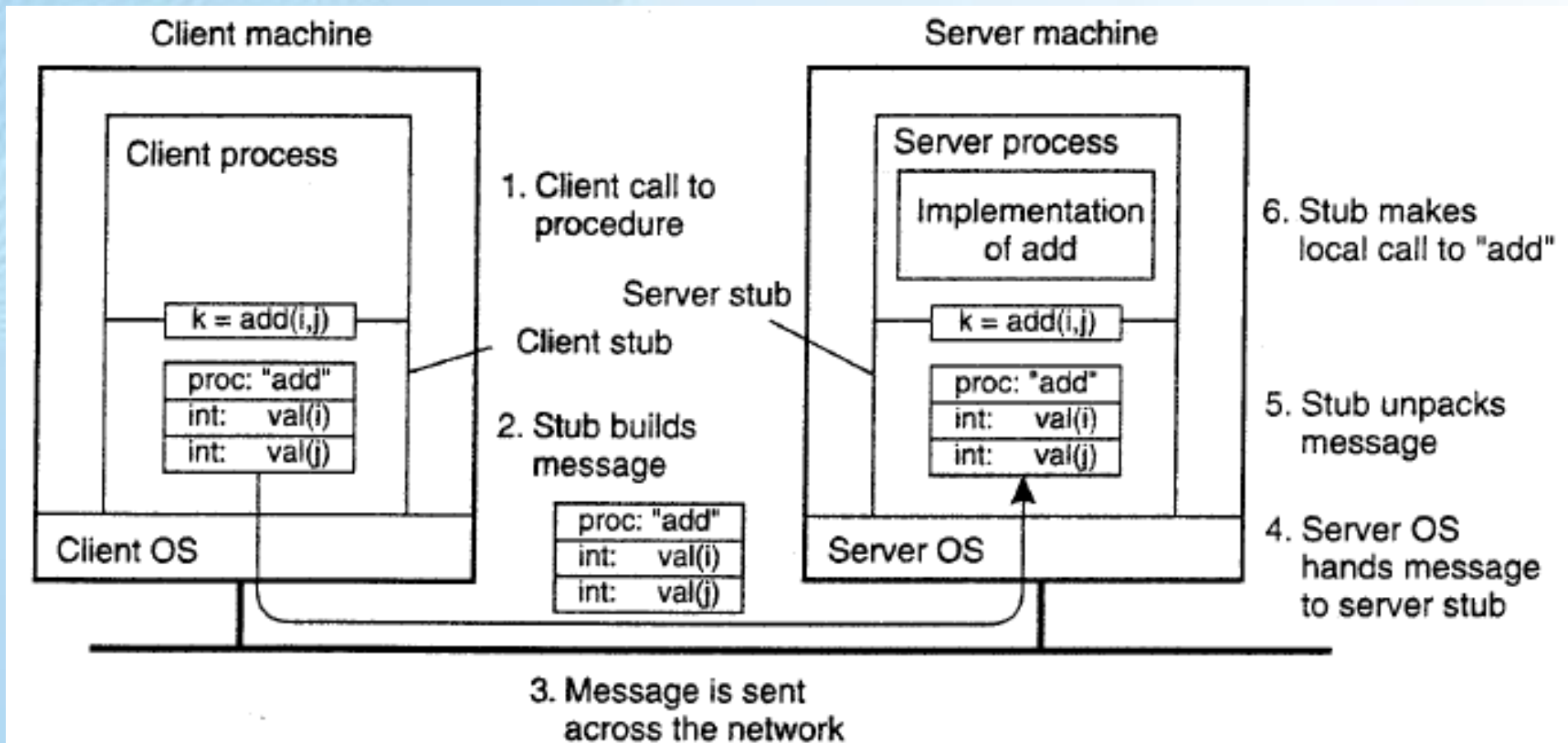


Figure 4-7. The steps involved in a doing a remote computation through RPC.

RPC Semantics

- Principle of RPC between a client and server program

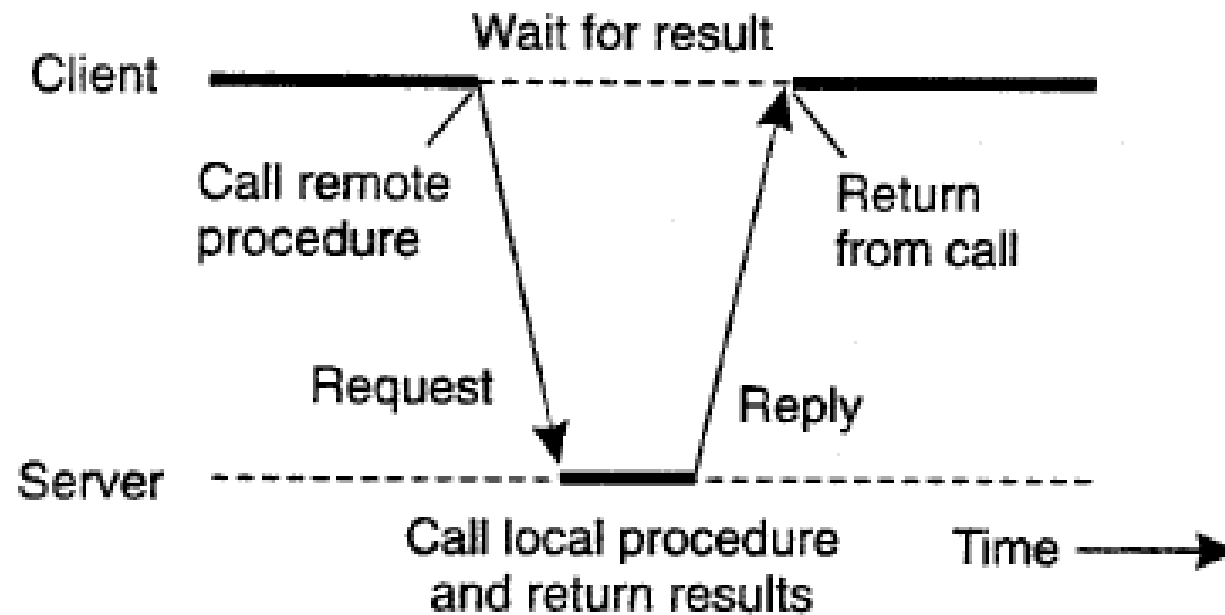


Figure 4-6. Principle of RPC between a client and server program.

RPC: Client and Server Stubs

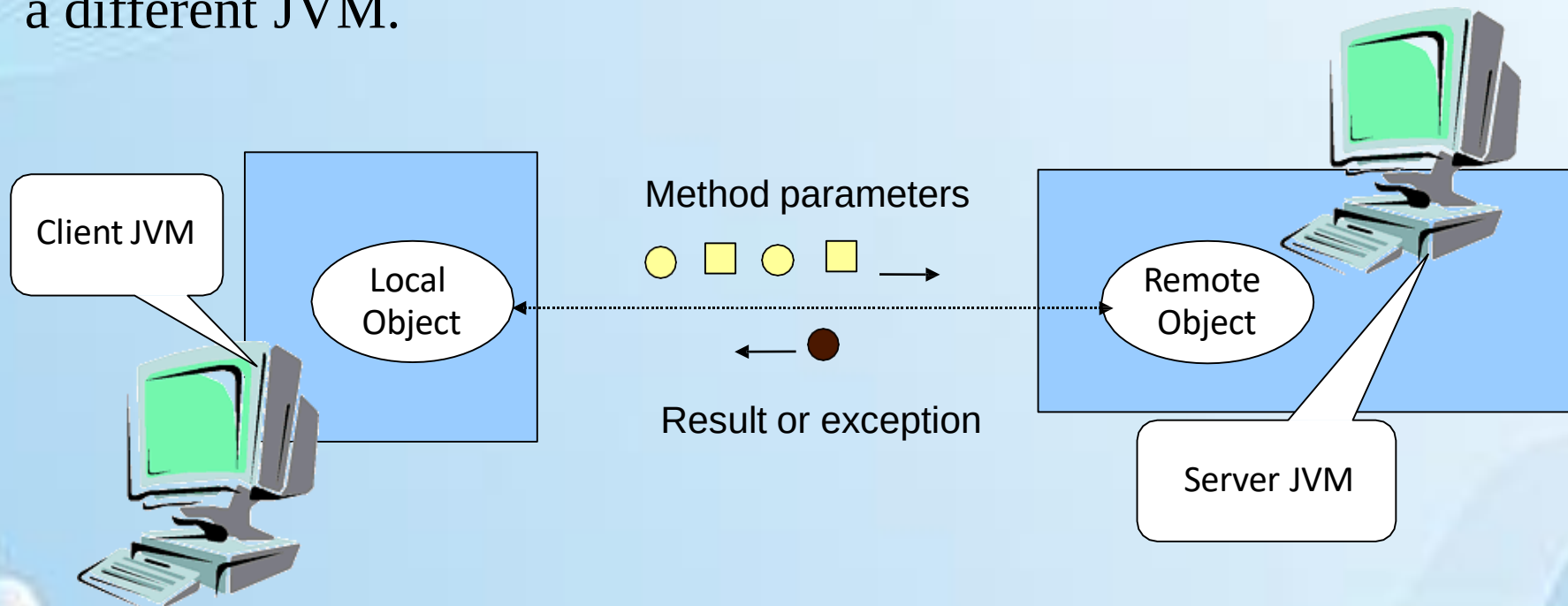
- Client makes procedure call(just like a local procedure call) to the client stub
- Stubs take care of packaging arguments and sending messages.
- Packaging parameters is called marshalling.
- Stub compiler generates stub automatically(simplifies programmer task)

Steps of a Remote Procedure Call

1. The client procedure calls the client stub in the normal way.
2. The client stub builds a message and calls the local OS.
3. The client's OS sends the message to the remote OS.
4. The remote OS gives the message to the server stub.
5. The server stub unpacks the parameters and calls the server.
6. The server does the work and returns the result to the stub.
7. The server stub packs it in a message and calls its server OS.
8. The server's OS sends the message to the client's OS.
9. The client's OS gives the message to the client stub.
10. The stub unpacks the result and returns to the client.

Remote Method Invocation(RMI)

- it is an expansion of the RPC mechanisms
- Java RMI allow one Java object to call methods on another Java object in a different JVM.



Remote Method Invocation(RMI)

- Java RMI allowed programmers to execute remote function class using the same semantics as local functions calls.

Local Machine (Client)

```
SampleServer remoteObject;  
int s;  
...  
s = remoteObject.sum(1,2);  
  
System.out.println(s);
```

1,2

3

Remote Machine (Server)

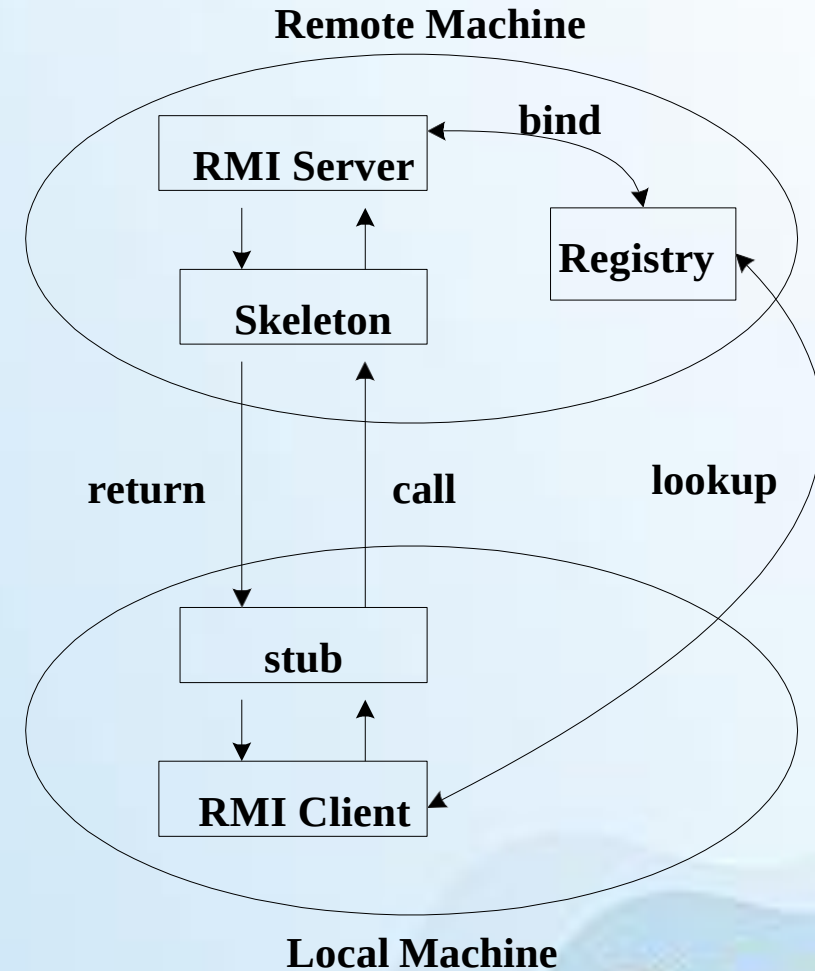
```
public int sum(int a,int b) {  
    return a + b;  
}
```

Terminology

- A **remote object** is an object on another computer
- The **client object** is the object making the request (sending a message to the other object)
- The **server object** is the object receiving the request
- As usual, “client” and “server” can easily trade roles (each can make requests of the other)
- The **rmiregistry** is a special server that looks up objects by name
 - Hopefully, the name is unique!
- **rmic** is a special compiler for creating **stub** (client) and **skeleton** (server) classes

The RMI Architecture

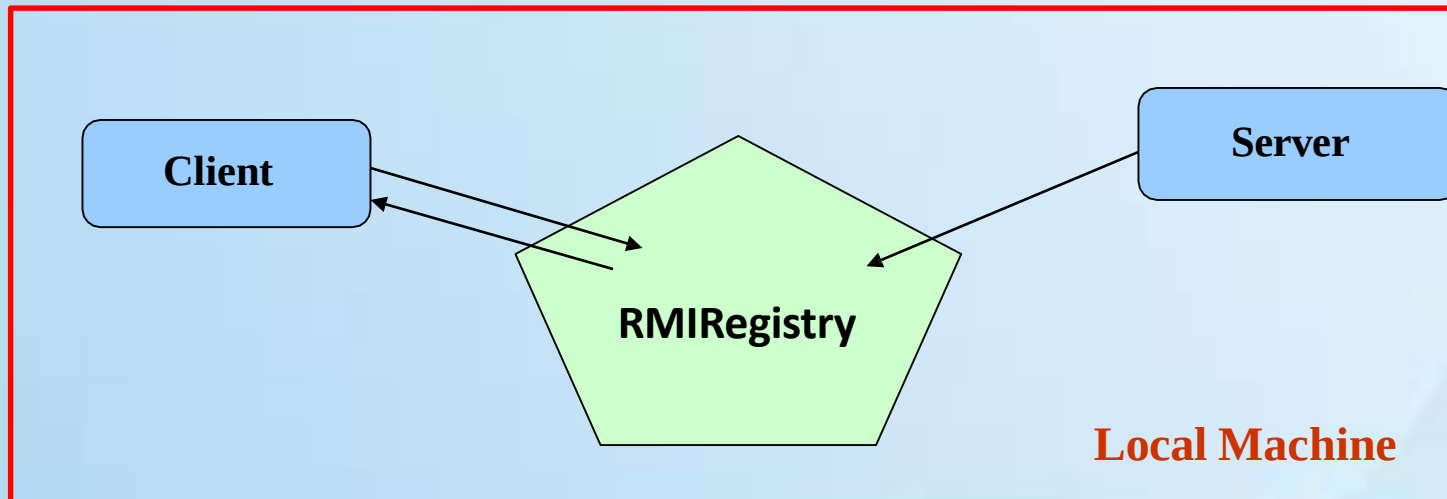
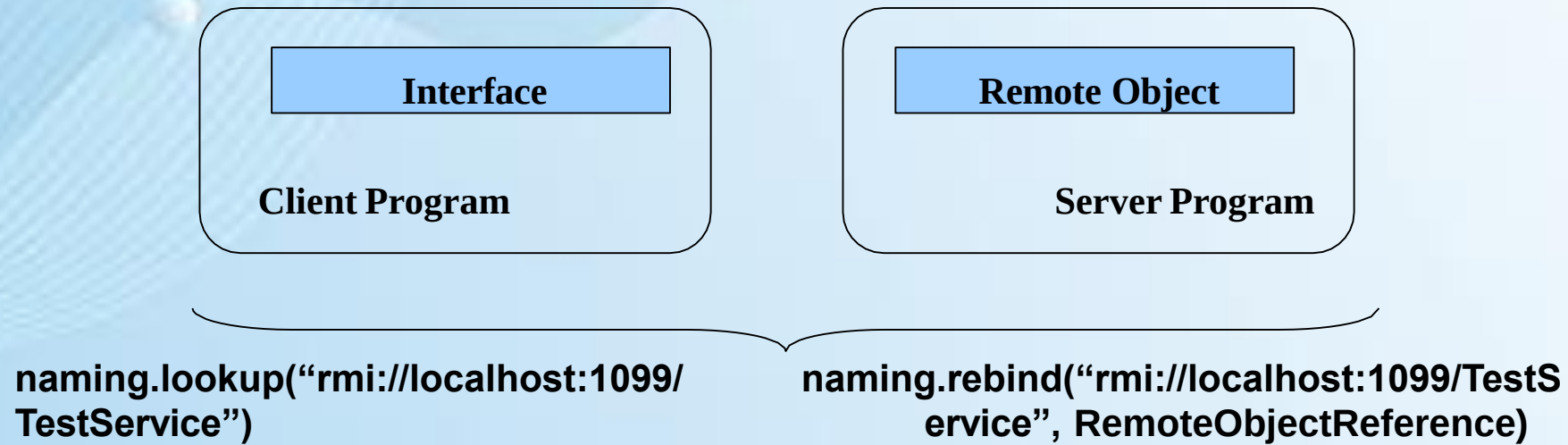
- The server must first bind its name to the registry
- The client lookup the server name in the registry to establish remote references.
- The Stub serializing the parameters to skeleton, the skeleton invoking the remote method and serializing the result back to the stub.



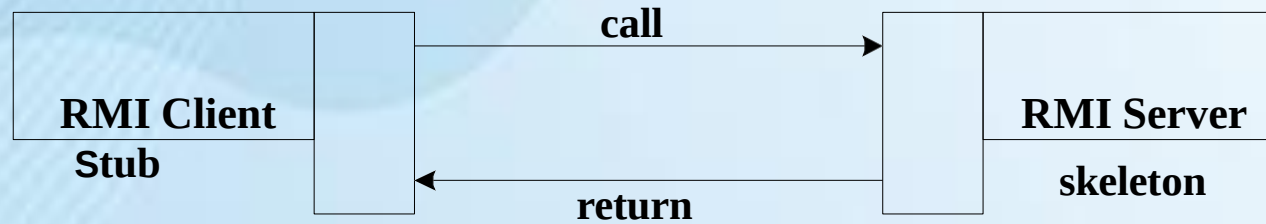
The RMI Registry

- The RMI Registry is a naming service
 - Separately Running service
 - Initiated using Java's “**rmiregistry**” tool
 - Server programs register remote objects
 - Give the object a name it can be found using
 - Client programs lookup object references that match this service name
- Registry names have a URL format
 - **rmi://<hostname>:<port>/<ServiceName>**
 - E.g. rmi://localhost:1099/CalculatorService
 - E.g. rmi://194.80.36.30:1099/ChatService

Lookup in Java RMI



The Stub and Skeleton



- A client invokes a remote method, the call is first forwarded to stub.
- The stub is responsible for sending the remote call over to the server-side skeleton
- The stub opening a socket to the remote server, marshaling the object parameters and forwarding the data stream to the skeleton.
- A skeleton contains a method that receives the remote calls, unmarshals the parameters, and invokes the actual remote object implementation.

Parameter Passing



- Parameter Passing in Java RMI is different from standard Java
 - *Reminder:* In Java, **primitives are passed by value**, Objects are passed by reference
- In Java RMI
 - **Objects and primitives are passed by value**
 - **Remote objects are passed by reference**
- You must be aware of the differences!
 - Otherwise you'll generate logical bugs that are difficult to identify



Parameter Passing

- RMI-Pass by Value
 - All ordinary objects and primitives are serialised and a copy is passed
 - Any changes to the copy do not affect the original
 - It is your job to ensure your Objects can be serialised!
- RMI-Pass by Reference
 - Remote Object is the parameter, a stub (reference) is sent
 - the stub is used to modify the object, the original object is modified

Thank You!!